



UNIVERSITY OF NOVI SAD
FACULTY OF SCIENCES

PROJECT IN INFORMATICS

INTERNSHIP AT ECLEPT

Full-Stack Movie Ticket Booking Web Application

Professor:

Vladimir Kurbalija

Student:

Andrej Rajkov

Novi Sad, 2026.

Table of Contents

Internship Assignment	3
1. Introduction	6
1.1 About the Project	6
1.2 Scope and Decisions	6
2. Approach and Learning Path	6
2.1 Resources and Role Models	7
2.2 Working Method	7
3. Technology Stack	8
3.1 Backend	8
3.2 Frontend	8
4. System Architecture	8
4.1 Authentication Flow	9
4.2 Project Structure	9
5. Database Design	9
5.1 Migrations with Alembic	10
6. Backend	11
6.1 API Endpoints	11
6.2 Authentication and Authorization	11
6.3 Validation and Security	12
6.4 Testing the API	12
7. Frontend	13
7.1 Public Browsing	13
7.2 Booking Flow	14
7.3 User Dashboard	14
7.4 Admin Panel	15
7.5 Theming	15
8. Deployment	16
9. Conclusion	16
References	17
Appendix	17
A. Project Links and Access	17
B. Acknowledgements	17

MovieMate

Description

MovieMate is a movie ticket booking system. The goal of this project is to simulate a real-world movie booking application, where users can easily navigate available films, select showtimes, and complete ticket reservations efficiently. The system should be designed with a user-friendly interface, ensuring seamless interactions for both guests and registered users. The system should ensure seat availability before confirming a booking. Additionally, users can choose specific seats if implemented as a bonus feature.

The platform consists of two main (**registered**) user roles: **Regular Users** and **Admins**. Regular users can browse movies and book tickets, while admins manage movies, auditoriums, and showtimes.

Best practices should be followed to prevent unauthorized access, data inconsistencies, and **overbooking issues**.

Project Scope & Core Features

1. User Authentication & Authorization

- Implement user authentication (sign-up, login) using a secure authentication mechanism.
- Use a token-based authentication system.
- Support different user roles:
 - Guests (Unregistered users): Can browse movies and view showtimes but cannot book tickets.
 - **Regular users**: Can browse movies, view showtimes, and book tickets.
 - **Admins**: Can manage movies, showtimes, and seating capacity.

2. Movie and Showtime Management

- Allow admins to add, update, and delete movies.
- Each movie can have multiple showtimes at different auditoriums.
- Ensure that movies contain relevant metadata.

3. Auditorium Management

- Each movie is shown in multiple auditoriums, each with a limited number of seats.
- Admins should be able to configure the seating capacity of each auditorium.

4. Booking system

- Guests can browse movies and showtimes but must register and log in to book tickets.
- Users can book tickets for a selected movie and showtime.
- Seats are automatically selected for the user.
- Ensure proper validation to prevent overbooking.
- Users should be able to view their bookings.

5. Security & Data Validation

- Ensure proper authentication and access control.
- Validate all inputs to prevent common vulnerabilities such as SQL injection and XSS.
- Restrict **unauthorized** users from modifying data.

Additional Points (Bonus)

- **Advanced Seat Selection:** Instead of being assigned a seat automatically, users can pick specific seats from an interactive seating layout.
- **Virtual Waiting Room:** If a show is in high demand, users can be placed in a queue before booking.
- **External Movie Database Integration:** Fetch additional movie metadata (ratings, actors, posters, etc.) from an external source.
- **Notifications:** Implement email or SMS notifications for booking confirmations.
- **Unit Tests:** Implement unit tests for key features to ensure reliability and maintainability.

Evaluation Criteria

- **Code Quality:** Clean, well-structured, and maintainable code with meaningful comments.
- **Functionality:** All core features implemented and working as expected.
- **Security:** Proper authentication, authorization, and data validation.
- **User Experience:** An intuitive and user-friendly interface.

We encourage creativity and independent decision-making regarding database structure and additional features. Best of luck!

1. Introduction

1.1 About the Project

MovieMate is the project I built during my internship at Eclept. The idea is simple to describe and surprisingly rich to build: a cinema booking system where visitors browse films and showtimes, registered users reserve seats and keep track of their bookings, and administrators manage the whole catalogue behind the scenes. The assignment, reproduced on the previous pages, left the technical choices open, so the application is the result of a series of decisions I made and will explain throughout this report.

The result is a full-stack web application. The backend is a REST API written in Python with FastAPI, backed by a PostgreSQL database. The frontend is a single-page application written in Angular. Both halves live in the same repository and are deployed live on the internet, so the project is not only a local demo but something anyone can open in a browser.

Live application: <https://andrej1011.github.io/eclipt-praksa/>

Source code: <https://github.com/andrej1011/eclipt-praksa>

1.2 Scope and Decisions

The brief lists a set of core features and a set of optional bonus features. I implemented all of the core features and made deliberate choices about the rest. Seats are assigned automatically by count rather than chosen from an interactive map. I did not build the virtual waiting room, since it solves a scale problem this project does not have. For richer movie data I integrated an external source, fetching posters and descriptions so the catalogue looks like a real cinema rather than a list of titles.

Three kinds of visitor exist, exactly as requested: guests who can browse but not book, registered users who can book and manage their reservations, and administrators who manage everything. Every restriction is enforced on the server, so the rules hold no matter what the browser tries to do.

2. Approach and Learning Path

I did not work on this alone. Throughout the internship I was guided by my mentor, Marko Šutić, a full-stack software engineer, and a lot of what I learned came directly from working with him day to day.

We met every morning for a short meeting. I would walk him through what I had done the day before, he would explain the new concepts I needed for the day ahead, and together we would set a plan for what I should build next. That rhythm kept me focused and meant I never sat stuck for long.

The actual work followed a simple loop. I would implement a piece of the project and open a pull request on GitHub. Marko would review it and leave comments, I would make the changes he suggested on the same branch, and once everything looked right we would merge it. Going through this review cycle over and over taught me more than any tutorial could. His experience was most valuable in showing me how to do the project properly: what a clean project structure should look like, and how to avoid the kind of beginner mistakes that are hard to spot on your own.

2.1 The First Week Foundations

The first week was dedicated to learning. I refreshed my understanding of object-oriented programming and the SOLID principles, went back over core database concepts, and brushed up on advanced Python concepts.

Once that groundwork was solid, I built a smaller practice project. The goal was narrow on purpose: learn how FastAPI works and set up a simple PostgreSQL database behind it. By the time I moved on to MovieMate, the framework and the database already felt familiar, and I could spend my attention on the project itself rather than on the basics.

2.2 External Resources

Two sources shaped how I worked. Telusko's FastAPI tutorials were my entry point into the framework and explained the fundamentals clearly. For structure and overall code organisation I followed ArjanCodes, whose videos and public example repository served as a model for how to split a project into routes, services, models, and schemas instead of piling everything into one file. That separation is visible throughout the backend.

2.3 Working Method

I built the backend first and then built the frontend on top of it. Rather than guessing at the shape of each endpoint, I confirmed it against the backend before wiring up the matching screen. The work moved in phases, each one a complete and testable slice: authentication, then public browsing, then booking, then the dashboard, then the admin panel, and finally polish and deployment. Working this way meant the application was always in a runnable state.

3. Technology Stack

The stack was chosen to be modern, widely used in industry, and a good learning for my career.

3.1 Backend

- **FastAPI** - a fast, modern Python web framework with automatic request validation and built-in interactive documentation.
- **SQLAlchemy** - the ORM that maps Python classes to database tables and keeps queries safe and expressive.
- **Pydantic** - validates and shapes every request and response, so malformed input is rejected before it reaches the logic.
- **Alembic** - manages database migrations, keeping the schema versioned and reproducible.
- **PostgreSQL** - the relational database that stores all of the application's data.
- **JWT and bcrypt** - token-based authentication and salted password hashing.
- **Docker** - packages the backend and database so the whole system runs the same way everywhere.

3.2 Frontend

- **Angular 22** - a complete framework for single-page applications, using standalone components and signals for state.

4. System Architecture

The application has three parts: the Angular single-page app that runs in the browser, the FastAPI server that exposes the REST API, and the PostgreSQL database that stores the data. The browser never talks to the database directly. It calls the API over HTTPS, sending a token with each request, and the API is the only thing that touches the database.

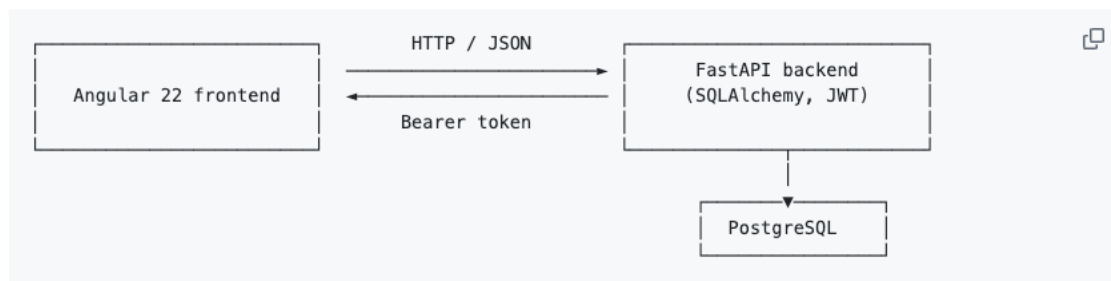


Figure 1: High-level system architecture

4.1 Authentication Flow

When a user logs in, the server checks the password against its stored bcrypt hash and returns a signed JSON Web Token. The frontend keeps that token and attaches it to every later request. The server validates the token on each call and reads the user's role from it, which is how it decides whether a request is allowed. Because the check happens on the server, a user cannot gain admin powers by editing anything in the browser.

4.2 Project Structure

The backend follows the separation I learned from ArjanCodes and my mentor's suggestions: routes handle HTTP, services hold the logic, models describe the tables, and schemas validate the data. The frontend mirrors this idea with a core layer of services and models, a shared layer of reusable pieces, and a feature layer for each screen.

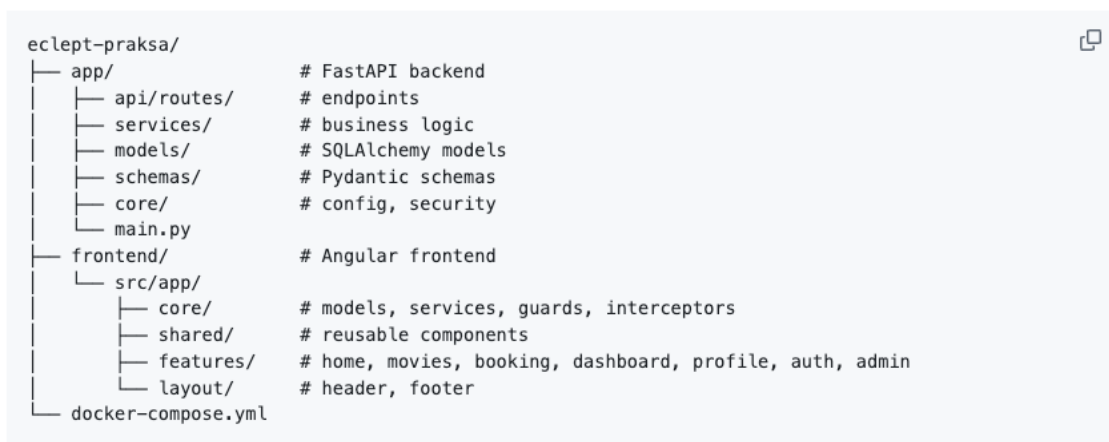


Figure 2: Backend and frontend folder structure

5. Database Design

The data model is the backbone of the application. Six main tables describe everything the system needs, plus one association table that links movies to their genres.

- **users** - accounts, including the hashed password and the role.
- **movies** - films and their metadata (title, duration, poster, description, release date).
- **genres** - categories, linked to movies through the movie_genres association table.
- **auditoriums** - screening rooms, each with a seating capacity.
- **showings** - a specific movie playing in a specific auditorium at a given time.
- **bookings** - a user's reserved seats for a showing.

The relationships read naturally: a user has many bookings, each booking belongs to one showing, a showing ties together one movie and one auditorium, and movies and genres form a many-to-many relationship. The diagram below shows the full schema.

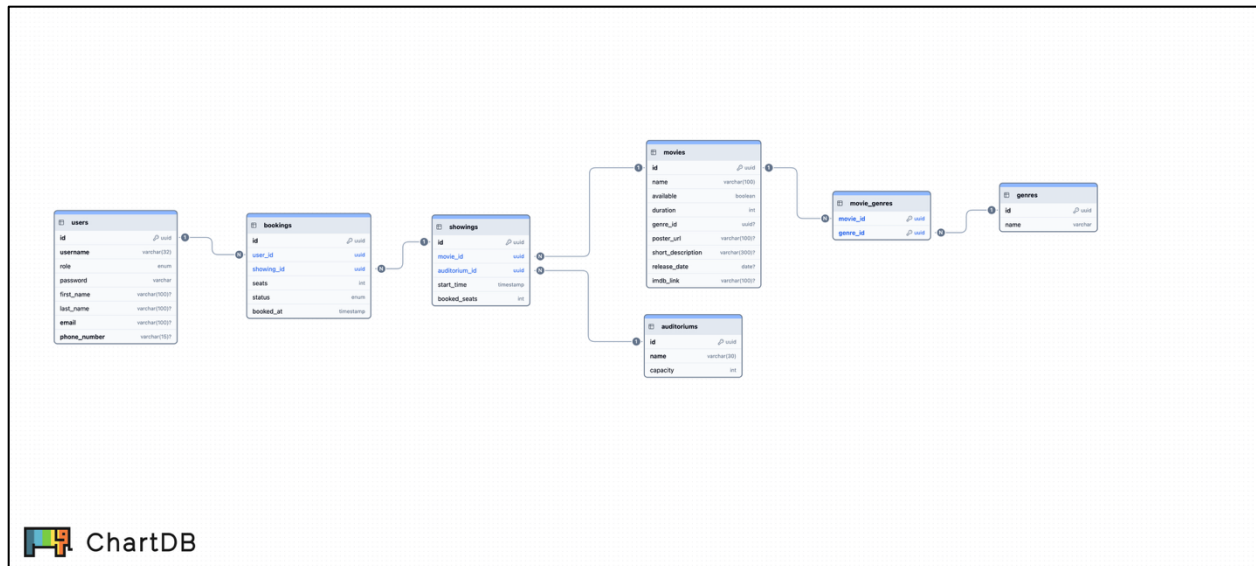


Figure 3: Database schema (entity-relationship diagram)

5.1 Migrations with Alembic

Rather than creating tables by hand, the schema is managed with Alembic migrations. Each change to the models becomes a versioned migration that can be applied to any database with a single command. This turned out to be one of the most valuable decisions in the project: when it came time to deploy, building the entire schema on a fresh cloud database was automatic rather than a manual chore.

6. Backend

The backend is a REST API. Every resource in the system has a clear set of endpoints, and the ones that change data are protected so that only the right users can reach them.

6.1 API Endpoints

The table below lists the main endpoints grouped by area. Endpoints marked as admin are restricted to administrators.

Area	Endpoints
Auth	POST /login, POST /register
Users	GET /users/me, PATCH /users/me, PATCH /users/me/password, GET /users (admin), DELETE /users/{id} (admin)
Movies	GET /movies, GET /movies/{id}, POST / PATCH / DELETE (admin)
Genres	GET /genres, POST / PATCH / DELETE (admin)
Auditoriums	GET /auditoriums, POST / PATCH / DELETE (admin)
Showings	GET /showings, POST / PATCH / DELETE (admin), POST /showings/{id}/cancel (admin)
Bookings	POST /bookings, GET /bookings/me, POST /bookings/{id}/cancel

6.2 Authentication and Authorization

Authentication uses JSON Web Tokens. On registration the password is hashed with bcrypt and only the hash is stored, never the plain text. On login the server verifies the password and issues a token that encodes the user and their role. Guards on the protected routes read that role and reject anyone who is not allowed in.

```

21 @router.post("/login", response_model=LoginResponse, status_code=status.HTTP_200_OK)
22 def login(data: LoginRequest, service: AuthService = Depends(get_auth_service)):
23     user, token = service.login(data)
24     return LoginResponse(user=user, access_token=token)
25
26 bearer_scheme = HTTPBearer()
27
28 def get_current_user(
29     creds: HTTPAuthorizationCredentials = Depends(bearer_scheme),
30     service: AuthService = Depends(get_auth_service),
31 ) -> User:
32     return service.get_user_by_token(creds.credentials)
33
34 def require_role(*roles: UserRole):
35     def checker(user: User = Depends(get_current_user)) -> User:
36         if user.role not in roles:
37             raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="You do not have permission to perform this action")
38         return user
39     return checker
40

```

Figure 4: Authentication: login, token issuing, and role checks

6.3 Validation and Security

Every incoming request is validated by Pydantic before it reaches any logic, so malformed or missing fields are rejected early with a clear error. Because all database access goes through SQLAlchemy rather than hand-written SQL strings, the application is not exposed to SQL injection. The booking logic is written to prevent overbooking: the server checks remaining capacity before confirming a reservation and refuses the request if not enough seats are left.

```
16 def create(self, user_id: UUID, data: BookingCreate) -> Booking:
17     showing = (
18         self._db.query(Showing)
19         .filter(Showing.id == data.showing_id)
20         .with_for_update()
21         .first()
22     )
23     if not showing:
24         raise HTTPException(status_code = status.HTTP_404_NOT_FOUND, detail = "Showing not found")
25
26     if showing.booked_seats + data.seats > showing.auditorium.capacity:
27         raise HTTPException(status_code = status.HTTP_409_CONFLICT, detail = "Not enough seats available")
28
29     showing.booked_seats += data.seats
30     booking = Booking(
31         user_id=user_id,
32         showing_id=data.showing_id,
33         seats=data.seats,
34         status=BookingStatus.active,
35     )
```

Figure 5: Booking validation that prevents overbooking

6.4 Testing the API

FastAPI generates interactive API documentation automatically. Throughout development I tested every endpoint directly from this built-in Swagger interface, available at the /docs path, by sending real requests and inspecting the responses before ever touching the frontend. It made verifying the backend quick and removed any guesswork about how an endpoint behaved.

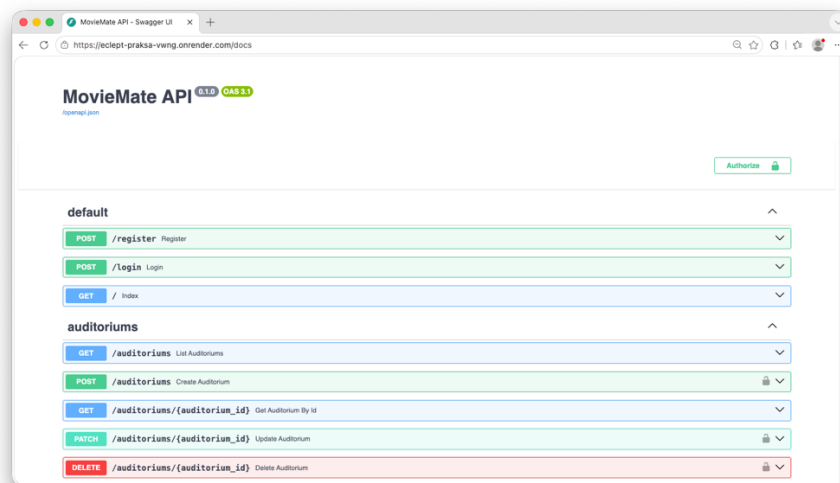


Figure 6: Interactive API documentation (Swagger UI at /docs)

7. Frontend

The frontend is a single-page application built with standalone Angular components and signals for state. A small set of typed services talks to the API, an interceptor attaches the authentication token to every request, and route guards keep protected and admin-only pages out of reach for those who should not see them.

7.1 Public Browsing

The home page presents a grid of films with a live search box that queries the server as the user types. The movie list adds genre, sorting, and ordering controls, and each film has a detail page showing its poster, information, and showtimes grouped by day. Sold-out showings are clearly marked so users know what is still available.

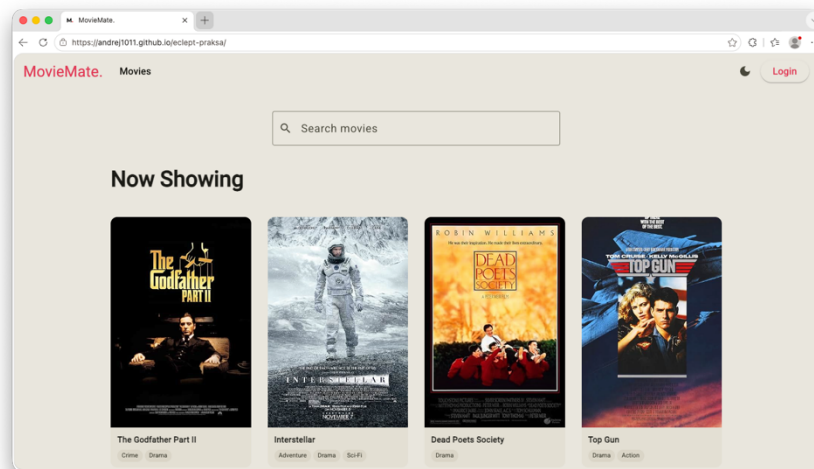


Figure 7: Home page with the movie grid and search

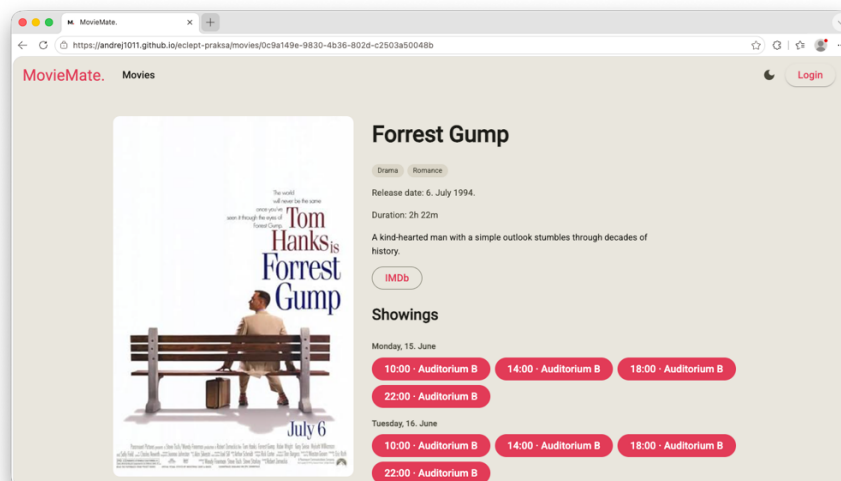


Figure 8: Movie detail page with showtimes grouped by day

7.2 Booking Flow

Booking a ticket is a short, focused flow. The user picks a showing, chooses how many seats they want within the remaining capacity, and confirms. If the showing fills up in the meantime, the server rejects the request and the interface explains what happened instead of failing silently.

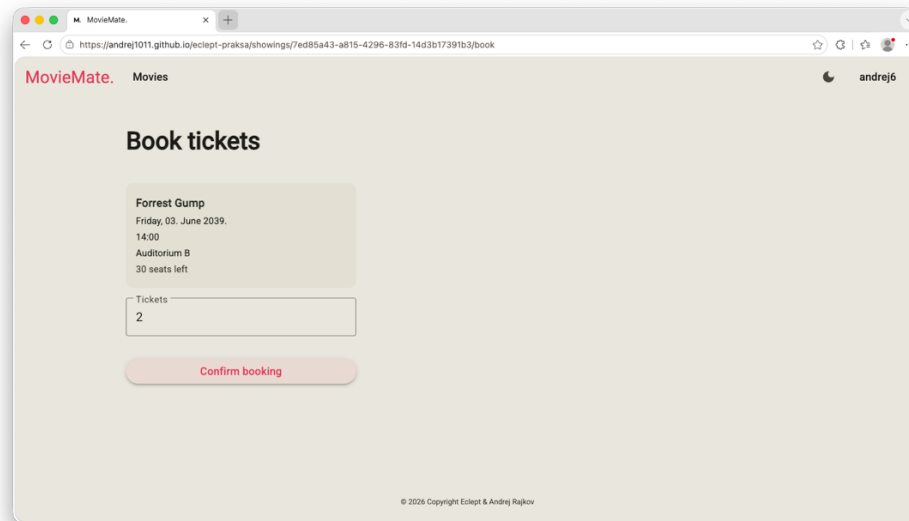


Figure 9: Booking page

7.3 User Dashboard

Each user has a dashboard that separates their bookings into upcoming and past, with the option to cancel a reservation. Cancelling frees the seats again, which is immediately reflected in the showing's availability.

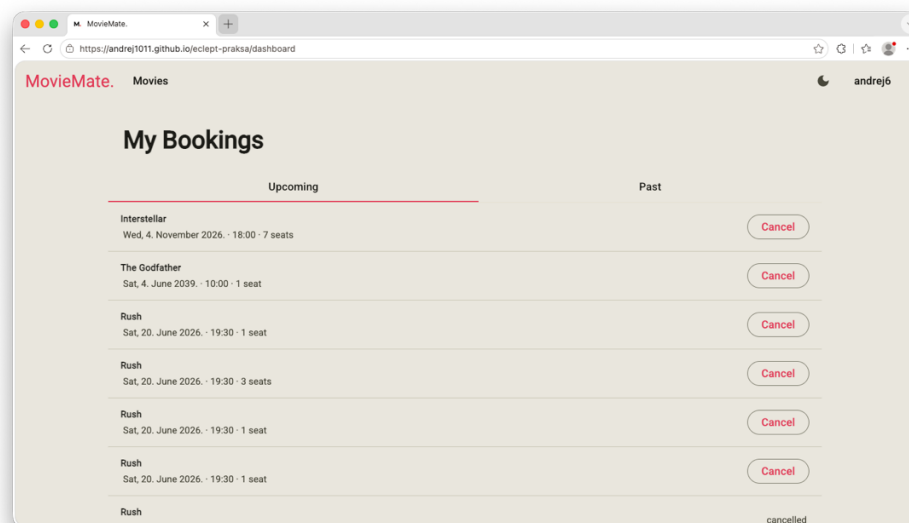


Figure 10: User dashboard with upcoming and past bookings

7.4 Admin Panel

The admin panel is the largest part of the frontend. It provides full create, read, update, and delete management for genres, auditoriums, movies, showings, and users, each following the same pattern of a table with dialogs for editing and confirmations for deletion. Scheduling showings uses calendar and time pickers, and a bulk tool can create many showings at once.

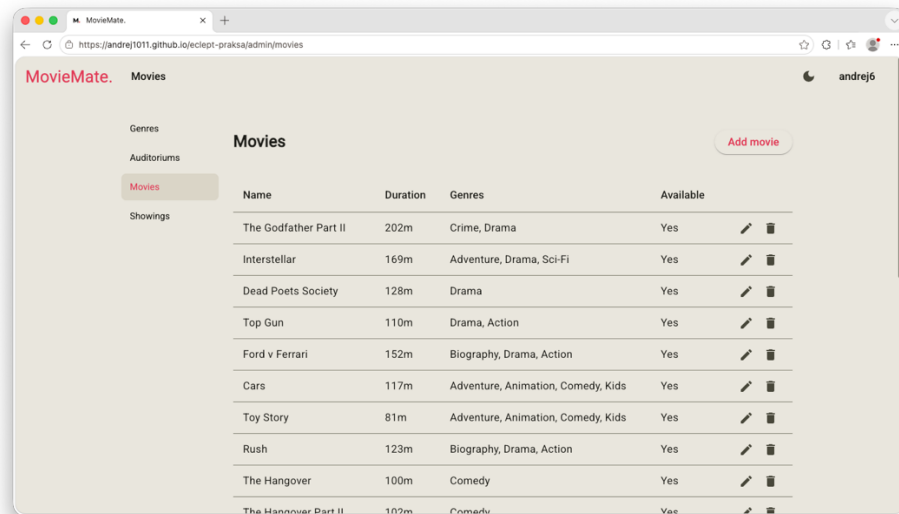


Figure 11: Admin panel: resource table and edit dialog

7.5 Theming

The application supports a light and a dark theme. The choice is remembered between visits and defaults to the visitor's system preference, so the interface feels at home on any device.

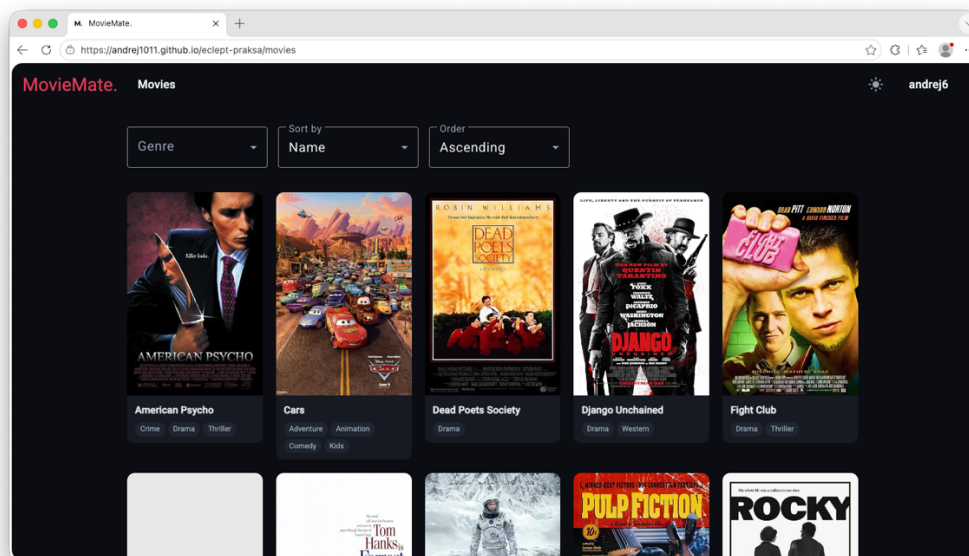


Figure 12: Light and dark theme

8. Deployment

A goal of the project was to put the application online, not just run it locally. Docker and Alembic made this far easier than it would otherwise have been. The backend and database are containerised, so the same setup that runs on my machine runs in the cloud, and Alembic builds the database schema automatically on startup.

The three parts are hosted on free services that fit a portfolio project. The Angular frontend is a static build served by GitHub Pages. The FastAPI backend runs as a container on Render. The PostgreSQL database is hosted on Neon. Together they give the application a public, shareable address at no cost.

9. Conclusion

MovieMate began as a request to build a booking system and grew into a complete full-stack exercise. I designed a database, built a secure REST API, hardened it by actually using it from a real client, created an Angular interface covering browsing, booking, a user dashboard, and a full admin panel, and finally deployed the whole thing to the public internet.

Along the way I learned how the pieces of a modern web application fit together, and how decisions on one side ripple to the other. There is clear room to grow: an interactive seat map, email confirmations for bookings, an endpoint that lets administrators change user roles from the interface, and an automated test suite would all be natural next steps. As it stands, the project meets the assignment in full and stands on its own as a working, deployed application.

References

- I. Telusko. *FastAPI Full Course*. YouTube. <https://youtu.be/uy4TPngK-8c>
- II. ArjanCodes. *Project structure tutorial*. YouTube. <https://www.youtube.com/watch?v=tc713anE3UY>
- III. ArjanCodes. Example repository used as a guide for organizing files: <https://github.com/ArjanCodes/examples/tree/main/2025/project>
- IV. FastAPI Official Documentation. <https://fastapi.tiangolo.com>
- V. SQLAlchemy Official Documentation (<https://www.sqlalchemy.org>)
- VI. Alembic Official Documentation (<https://alembic.sqlalchemy.org>)
- VII. Angular Official Documentation (<https://angular.dev>)
- VIII. Render Documentation (<https://render.com/docs>)
- IX. Neon Documentation (<https://neon.tech/docs>)

(All links accessed June 2026.)

Appendix

A. Project Links and Access

Live application: <https://andrej1011.github.io/eclipt-praksa/>

Source code: <https://github.com/andrej1011/eclipt-praksa>

Demo administrator: username “admin”, password “admin” (for evaluation).

B. Acknowledgements

I would like to thank the team at Eclipt for the opportunity to work on this project during my internship, and for being trusted to make my own technical decisions, which is what made it such a valuable learning experience.

My thanks go especially to my mentor, Marko Šutić, whose guidance shaped this project from start to finish. Our daily discussions, his patient explanations of new concepts, and his detailed reviews of every pull request taught me far more than I expected. He showed me how to structure a project properly and helped me catch the kind of mistakes that are hard to notice on your own.
